Temporal Gait Transformer: Classifying Walking Activities from Smart Insole Data

Sebastian Kot (sebastian.kot@city.ac.uk) Mohammad Karimi (mohammad.karimi.3@city.ac.uk)
Wadah Abdelaziz (Wadah.abdelaziz@city.ac.uk) Mohammad Aqib Iqbal (aqib.iqbal@city.ac.uk)

March 23, 2026

The objective of this project was to train a deep-learning classifier that can identify walking styles from a smart insole device. After analysing the provided MLP baseline code, we decided to implement an architecture better suited to temporal data: a small CNN-Transformer hybrid.

The MLP baseline takes a variable-length window of 48-feature rows (left + right foot) and flattens them into a single input vector. This removes any possibility of temporal analysis within the gait cycle. Our architecture treats each timestep as a separate token. A two-layer 1D convolutional stem first extracts local patterns (heel-strike, toe-off phases), then sinusoidal positional encodings are added so the model knows the ordering of timesteps. Two Transformer encoder layers then compute QKV attention scores, allowing the model to learn interactions across the full gait cycle, followed by a feed-forward network (GeLU activation, 4× dimension expansion). We mean-pool the resulting token representations and pass them through layer normalisation and a linear layer that outputs 5 logits, one per activity class.

Training uses cross-entropy loss with label smoothing. Class imbalances (e.g. Swaying has roughly twice as many windows as Stepping) are handled by weighting the loss by inverse class frequency and using a weighted random sampler with the same inverse probabilities. We optimise with AdamW (decoupled weight decay). Five augmentation transforms (Gaussian jitter, per-channel scaling, channel dropout, temporal masking, random time shift) are applied during training to reduce overfitting to the small number of training subjects. All data splits are done at the file level so overlapping windows from the same recording never appear in both training and validation sets. Further architectural and training details are provided in Appendix A.

For future model training, it would be best to increase the number of attention heads and increase dimensionality of models, assuming there is much more available training data. Current data is limited to 4 sessions, and there is a mismatch in frequency causing trouble in the classifier.

The main parameters used: window size = **80** (1.33 s at 60 Hz), stride = **30**, batch size = **64**, max epochs = **150**, patience = **80**, learning rate = 5×10^{-4}, d_{model} = **32**, **2** attention heads, **2** encoder layers, dropout = **0.3**.

Parameter	What we tried	Final	Why we kept final choice
Window size W	50, 80	**80**	Captures around a full gait cycle and improved class separation for temporally similar classes.
Stride S	10, 20, 30	**30**	Better compute vs overlap trade-off while keeping enough windows.
d_{model}	32, 64	**32**	Smaller embedding reduced overfitting with limited subjects and kept training stable.
Heads / FFN	2/128, 4/256	**2/128**	Similar validation, lower complexity.
Dropout	0.1, 0.3	**0.3**	More stable validation on unseen files.
Mixup α	0.0, 0.2	**0.2**	Slight validation gain on this run.

Table 1: Main hyperparameter choices, alternatives tested, and final rationale.

Validation Metric	Value
Best epoch	29 / 150
Train loss / accuracy	0.7036 / 55.83%
Val loss / accuracy	0.6154 / 93.29%
Macro precision	0.88
Macro recall	0.85
Macro F1	0.85
Weighted F1	0.92
Validation support	1864 windows

Table 2: Best validation run summary and final metrics.

Class	P	R	F1	n
Normal_walking	0.82	0.98	0.89	295
Injury_walking	0.96	1.00	0.98	401
Stepping	0.68	0.30	0.41	131
Swaying	0.99	0.96	0.98	741
Jumping	0.94	1.00	0.97	296

Table 3: Per-class validation metrics at the best checkpoint (Val accuracy 93.29%).

Result interpretation. The main weakness is Stepping recall (0.30), while other classes are strong (F1 0.89 to 0.98), this is because of limited data for this class. The relatively low train accuracy (55.83%) is expected with mixup because labels are blended during optimisation. In this setting, validation metrics are more informative than hard-label train accuracy.

Reproducibility. Code: github.com/sebinkooooo/mech. Training run: Google

Drive notebook. The full implementation used in this report is included in Appendix B.

Appendix note. Appendix A and Appendix B begin on the next page.

Appendix

A Model Specification (condensed more on github /paper)

A.1 Input and preprocessing

Each sample is a window of shape 80×48:

- 48 features per timestep = 24 per foot.
- Per foot: 18 pressure sensors (`ele_0` to `ele_17`), 3 accelerometer axes (`ele_18`, `ele_19`, `ele_20`), and 3 Euler orientation angles (`ele_22`, `ele_23`, `ele_24`).
- Left and right streams are merged by nearest timestamp match ($\delta t < 0.02\,\mathrm{s}$), then resampled to 60 Hz.
- Sliding windows use $W = 80$, stride $S = 30$ (62.5% overlap).
- Normalisation is two-stage: global z-score (fit on train only), then per-window z-score.

A.2 Exact TGT architecture

The final classifier is a CNN-Transformer hybrid:

- **Input:** $(B, 80, 48)$.
- **Conv stem:** two 1D conv blocks along time:
  - Conv1d($48 \to 32$, kernel 7, padding 3) + BatchNorm + GELU + Dropout(0.3)
  - Conv1d($32 \to 32$, kernel 5, padding 2) + BatchNorm + GELU + Dropout(0.3)
- **Positional encoding:** fixed sinusoidal encoding added to each of the 80 tokens.
- **Transformer encoder:** 2 layers, 2 heads, $d_{\text{model}} = 32$, FFN hidden size 128, activation GELU, dropout 0.3.
- **Sequence aggregation:** mean pooling across all 80 token outputs (no CLS token).
- **Head:** LayerNorm(32) + Dropout(0.3) + Linear($32 \to 5$ logits).

A.3 Training configuration and key choices

- **Loss:** cross-entropy with class weights (inverse class frequency) and label smoothing 0.1.
- **Sampler:** WeightedRandomSampler to oversample minority classes.
- **Optimiser:** AdamW, learning rate 5×10^{-4}, weight decay 10^{-3}.
- **Schedule:** cosine annealing over 150 epochs; early stopping patience 80.
- **Stability:** gradient clipping (max norm 1.0).
- **Augmentation:** Gaussian jitter (std 0.15), per-channel scaling [0.5, 1.5], channel dropout (0.2), temporal masking (up to $W/7$), and random circular time shift (up to $\pm W/8$).
- **Mixup:** $\alpha = 0.2$ for the training run attached in this report.
- **Split strategy:** file-level train/validation split to prevent leakage from overlapping windows.

B Full Source Code (functions)

```python
"""
sebi_v2.py  -  Temporal Gait Transformer (TGT) utilities
=======================================================
Same as sebi.py but uses Euler angles (ele_22/23/24 = roll/pitch/yaw)
instead of quaternions (ele_30-33) to match the standard MLP template
preprocessing.  This gives 48 input features per timestep (24 per foot)
instead of 50, ensuring the exported .pth is compatible with the
standard evaluation pipeline.
"""

import os
```

```python
import math
import numpy as np
import pandas as pd
from tqdm import tqdm

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

#  non-interactive matplotlib so headless / subprocess works
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler, MinMaxScaler


#
#  DATA  PREPROCESSING
#

def get_frame_rate(data):
    avg_interval = data["timestamp"].diff().mean()
    return 1 / avg_interval if avg_interval > 0 else None


def downsample_to_60Hz(data, target_fps=60):
    total_time = data["timestamp"].iloc[-1] - data["timestamp"].iloc[0]
    total_frames = len(data)
    current_fps = total_frames / total_time if total_time > 0 else None
    if current_fps and current_fps > target_fps:
        target_frame_count = int(total_time * target_fps)
        indices = np.linspace(0, total_frames - 1, target_frame_count, dtype=int)
        return data.iloc[indices].reset_index(drop=True)
    return data


def upsample_to_60Hz(data, target_fps=60):
    target_interval = 1.0 / target_fps
    new_ts = np.arange(data["timestamp"].min(), data["timestamp"].max(), target_interval)
    orig = data.set_index("timestamp")
    combined = orig.reindex(orig.index.union(new_ts)).sort_index()
    combined = combined.interpolate(method="index")
    data_interp = combined.reindex(new_ts).reset_index()
    data_interp.rename(columns={"index": "timestamp"}, inplace=True)
    return data_interp


def insole_process(file_path):
    """Read a raw insole CSV, merge L/R feet, resample to 60 Hz."""
    target_fps = 60
    data = pd.read_csv(file_path).dropna()
    left = data[data["ele_36"] == 1]
    right = data[data["ele_36"] == 0]

    # 0-17: pressure, 18-20: accelerometer, 22-24: Euler angles (roll/pitch/yaw)
    cols = [f"ele_{i}" for i in range(18)] + [
        "ele_18", "ele_19", "ele_20",
        "ele_22", "ele_23", "ele_24",
    ]
    left = left[["timestamp"] + cols]
    right = right[["timestamp"] + cols]

    left_fps = get_frame_rate(left)
    right_fps = get_frame_rate(right)

    if left_fps < right_fps:
        base, high, suffixes = left, right, ("_left", "_right")
    else:
        base, high, suffixes = right, left, ("_right", "_left")

    merged = pd.merge_asof(
        base.drop_duplicates(subset=["timestamp"]).sort_values("timestamp").reset_index(drop=True),
        high.drop_duplicates(subset=["timestamp"]).sort_values("timestamp").reset_index(drop=True),
        on="timestamp",
```

```python
        direction="nearest",
        suffixes=suffixes,
        tolerance=0.02,
    )

    if any(c.endswith("_right") for c in merged.columns[1:]):
        lcols = [c for c in merged.columns if "_left" in c]
        rcols = [c for c in merged.columns if "_right" in c]
        merged = merged[["timestamp"] + lcols + rcols]

    merged = merged.dropna()
    fps = get_frame_rate(merged)
    if fps < target_fps:
        return upsample_to_60Hz(merged, target_fps)
    if fps > target_fps:
        return downsample_to_60Hz(merged, target_fps)
    return merged


#  windowing

FEATURE_COLUMNS = [
    # Left pressure (18)
    *[f"ele_{i}_left" for i in range(18)],
    # Left accelerometer (3)
    "ele_18_left", "ele_19_left", "ele_20_left",
    # Left Euler angles (3): roll, pitch, yaw
    "ele_22_left", "ele_23_left", "ele_24_left",
    # Right pressure (18)
    *[f"ele_{i}_right" for i in range(18)],
    # Right accelerometer (3)
    "ele_18_right", "ele_19_right", "ele_20_right",
    # Right Euler angles (3): roll, pitch, yaw
    "ele_22_right", "ele_23_right", "ele_24_right",
]

LABEL_MAPPING = {
    "Normal_walking": 0,
    "Injury_walking": 1,
    "Stepping": 2,
    "Swaying": 3,
    "Jumping": 4,
}

CLASS_NAMES = ["Normal_walking", "Injury_walking", "Stepping", "Swaying", "Jumping"]


def load_and_window(data_dir, feature_columns, label_mapping,
                    window_size, stride):
    """
    Walk *data_dir* (expects sub-folders whose names contain a label key),
    preprocess every CSV, and return windowed numpy arrays.

    Returns (features, labels, file_ids) as numpy arrays.
    ``file_ids`` maps each window back to the source CSV so that
    train/val/test splits can be made at the *file* level, preventing
    data leakage from overlapping windows.
    """
    all_features, all_labels, all_file_ids = [], [], []
    if not os.path.isdir(data_dir):
        raise FileNotFoundError(f"{data_dir} does not exist")

    file_counter = 0
    for folder_name in sorted(os.listdir(data_dir)):
        folder_path = os.path.join(data_dir, folder_name)
        if not os.path.isdir(folder_path):
            continue
        for keyword, label_id in label_mapping.items():
            if keyword in folder_name:
                csvs = [f for f in os.listdir(folder_path) if f.endswith(".csv")]
                for fname in tqdm(csvs, desc=f"{keyword}", unit="file"):
                    fpath = os.path.join(folder_path, fname)
                    try:
                        feats = insole_process(fpath)[feature_columns]
                    except Exception as e:
                        print(f"  WARNING skipping {fname}: {e}")
                        continue
                    n = feats.shape[0]
```

```
                    for s in range(0, n - window_size + 1, stride):
                        all_features.append(feats.iloc[s : s + window_size].values)
                        all_labels.append(label_id)
                        all_file_ids.append(file_counter)
                    file_counter += 1
                break  # matched keyword; no need to check remaining

    return (np.array(all_features, dtype=np.float32),
            np.array(all_labels, dtype=np.int64),
            np.array(all_file_ids, dtype=np.int64))


def split_by_file(file_ids, train_ratio=0.8, val_ratio=0.1, seed=42):
    """
    Split window indices so that *all* windows from the same source CSV
    stay in the same partition.  This prevents data leakage caused by
    overlapping sliding windows ending up in different splits.

    Returns (train_idx, val_idx, test_idx) as numpy index arrays.
    """
    rng = np.random.RandomState(seed)
    unique_files = np.unique(file_ids)
    rng.shuffle(unique_files)

    n_files = len(unique_files)
    n_train = int(train_ratio * n_files)
    n_val = int(val_ratio * n_files)

    train_files = set(unique_files[:n_train].tolist())
    val_files = set(unique_files[n_train:n_train + n_val].tolist())
    test_files = set(unique_files[n_train + n_val:].tolist())

    train_idx = np.where(np.isin(file_ids, list(train_files)))[0]
    val_idx = np.where(np.isin(file_ids, list(val_files)))[0]
    test_idx = np.where(np.isin(file_ids, list(test_files)))[0]

    return train_idx, val_idx, test_idx


#
#  DATASET
#

class GaitDataset(Dataset):
    """
    Parameters
    ----------
    features : ndarray (N, W, F)
    labels   : ndarray (N,)
    scaler   : fitted sklearn scaler, optional
        If provided, this scaler is used to *transform* (not fit) the data.
        If None and ``standardize`` or ``normalize`` is True, a new scaler
        is fitted on this data and stored in ``self.scaler``.
    standardize : bool
        Fit / apply StandardScaler (z-score) across all windows.
    normalize : bool
        Fit / apply MinMaxScaler ([0, 1]) across all windows.
    window_norm : bool
        Z-score each window independently (per feature, across timesteps).
        Removes subject-specific baselines so the model only sees relative
        temporal patterns.  Applied AFTER any global scaler.
    augment : bool
        Enable training-time augmentation (noise, scaling, time mask).
    """

    def __init__(self, features, labels,
                 scaler=None, standardize=False, normalize=False,
                 window_norm=False, augment=False):
        n, w, f = features.shape
        flat = features.reshape(-1, f)

        if scaler is not None:
            flat = scaler.transform(flat)
            self.scaler = scaler
        elif standardize:
            sc = StandardScaler().fit(flat)
            flat = sc.transform(flat)
            self.scaler = sc
```

```python
        elif normalize:
            sc = MinMaxScaler().fit(flat)
            flat = sc.transform(flat)
            self.scaler = sc
        else:
            self.scaler = None

        features = flat.reshape(n, w, f)
        self.window_norm = window_norm
        self.augment = augment

        if window_norm:
            mean = features.mean(axis=1, keepdims=True)
            std = features.std(axis=1, keepdims=True) + 1e-8
            features = (features - mean) / std

        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        x = self.features[idx]
        if self.augment:
            x = self._augment(x)
        return x, self.labels[idx]

    @staticmethod
    def _augment(x):
        """Aggressive augmentation to improve cross-subject generalisation."""
        w, f = x.shape

        # 1. Gaussian noise - stronger to blur subject-specific patterns
        x = x + torch.randn_like(x) * 0.15

        # 2. Per-channel random scaling - simulates different body weight,
        #    sensor sensitivity, and foot-sensor contact across subjects
        scale = 0.5 + 1.0 * torch.rand(f)          # [0.5, 1.5]
        x = x * scale

        # 3. Channel dropout - randomly zero out ~20% of features to
        #    prevent reliance on any single sensor
        chan_mask = torch.rand(f) > 0.2              # keep 80%
        x = x * chan_mask.float()

        # 4. Time masking - zero out a contiguous block (up to ~15% of window)
        mask_len = torch.randint(1, max(2, w // 7), (1,)).item()
        start = torch.randint(0, max(1, w - mask_len), (1,)).item()
        x[start:start + mask_len] = 0.0

        # 5. Random time shift - circularly roll the window by a few steps
        #    to prevent the model from relying on absolute position
        shift = torch.randint(-w // 8, w // 8 + 1, (1,)).item()
        x = torch.roll(x, shifts=shift, dims=0)

        return x


def compute_class_weights(labels, num_classes):
    """Return inverse-frequency weights as a float32 tensor (for CrossEntropyLoss)."""
    counts = np.bincount(labels, minlength=num_classes).astype(np.float64)
    counts = np.maximum(counts, 1.0)
    weights = 1.0 / counts
    weights /= weights.sum()
    weights *= num_classes
    return torch.tensor(weights, dtype=torch.float32)


def make_sample_weights(labels):
    """Per-sample weights for WeightedRandomSampler (inverse class frequency)."""
    counts = np.bincount(labels)
    class_weight = 1.0 / np.maximum(counts, 1).astype(np.float64)
    return torch.tensor(class_weight[labels], dtype=torch.float64)


#
#  MODEL  -  Temporal Gait Transformer
```

```python
#

class TGTModel(nn.Module):
    """
    CNN-Transformer hybrid for gait classification.

    Architecture:
      1.  1D CNN stem (two conv layers) extracts local temporal patterns
          like heel-strike and toe-off phases - these are more subject-
          invariant than raw sensor values.
      2.  + sinusoidal positional encoding
      3.  N x Transformer encoder layers capture longer-range gait dynamics
      4.  Mean pooling over timesteps (more robust than CLS token with
          limited training subjects)
      5.  Classification head with LayerNorm + dropout
    """

    def __init__(self, num_features, window_size, num_classes=5,
                 d_model=64, num_heads=4, num_layers=2,
                 dim_feedforward=256, dropout=0.3):
        super().__init__()
        self.d_model = d_model

        # 1D CNN stem - local temporal feature extraction
        self.conv_stem = nn.Sequential(
            nn.Conv1d(num_features, d_model, kernel_size=7, padding=3),
            nn.BatchNorm1d(d_model),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Conv1d(d_model, d_model, kernel_size=5, padding=2),
            nn.BatchNorm1d(d_model),
            nn.GELU(),
            nn.Dropout(dropout),
        )

        pe = self._make_pe(window_size, d_model)
        self.register_buffer("pe", pe, persistent=False)

        enc_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=num_heads,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=True,
            activation="gelu",
        )
        self.encoder = nn.TransformerEncoder(enc_layer, num_layers=num_layers)

        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Dropout(dropout),
            nn.Linear(d_model, num_classes),
        )

        self._init_weights()

    #  helpers

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv1d):
                nn.init.kaiming_normal_(m.weight, nonlinearity="linear")
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.xavier_uniform_(m.weight)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)

    @staticmethod
    def _make_pe(length, d_model):
        pe = torch.zeros(length, d_model)
        pos = torch.arange(length, dtype=torch.float).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2, dtype=torch.float)
                        * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        return pe
```

```python

    #  forward

    def forward(self, x):
        """x: (B, window_size, num_features) -> logits (B, num_classes)"""
        x = x.transpose(1, 2)                    # (B, F, W) for Conv1d
        x = self.conv_stem(x)                     # (B, d, W)
        x = x.transpose(1, 2)                    # (B, W, d)
        x = x + self.pe.unsqueeze(0)             # positional encoding
        x = self.encoder(x)                       # (B, W, d)
        x = x.mean(dim=1)                        # mean pool -> (B, d)
        return self.head(x)                       # (B, C)


#
#  TRAINING  /  EVALUATION  HELPERS
#

def train_one_epoch(model, loader, criterion, optimizer, device,
                    max_grad_norm=1.0, mixup_alpha=0.0):
    """
    If mixup_alpha > 0, applies mixup: each mini-batch is blended with a
    shuffled copy of itself using a Beta(alpha, alpha) mixing coefficient.
    The loss is computed against both original and shuffled targets,
    weighted by the mixing coefficient.
    """
    model.train()
    loss_sum, correct, total = 0.0, 0, 0
    use_mixup = mixup_alpha > 0.0

    for x, y in loader:
        x, y = x.to(device), y.to(device)

        if use_mixup:
            lam = np.random.beta(mixup_alpha, mixup_alpha)
            idx = torch.randperm(x.size(0), device=device)
            x = lam * x + (1 - lam) * x[idx]
            y_a, y_b = y, y[idx]

        optimizer.zero_grad()
        logits = model(x)

        if use_mixup:
            loss = lam * criterion(logits, y_a) + (1 - lam) * criterion(logits, y_b)
        else:
            loss = criterion(logits, y)

        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), max_grad_norm)
        optimizer.step()
        loss_sum += loss.item()
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return loss_sum / len(loader), 100.0 * correct / total


@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    loss_sum, correct, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss = criterion(logits, y)
        loss_sum += loss.item()
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return loss_sum / len(loader), 100.0 * correct / total


@torch.no_grad()
def predict_all(model, loader, device):
    """Return (all_preds, all_labels) as numpy arrays."""
    model.eval()
    preds, labels = [], []
    for x, y in loader:
        x = x.to(device)
        preds.append(model(x).argmax(1).cpu().numpy())
```

```python
        labels.append(y.numpy())
    return np.concatenate(preds), np.concatenate(labels)


#
#  PLOTTING  HELPERS
#

def plot_training_curves(train_losses, val_losses, train_accs, val_accs,
                         save_path):
    epochs = range(1, len(train_losses) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

    ax1.plot(epochs, train_losses, label="Train Loss", color="royalblue")
    ax1.plot(epochs, val_losses, label="Val Loss", color="crimson")
    ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
    ax1.set_title("Loss Curve"); ax1.legend(); ax1.grid(alpha=0.3)

    ax2.plot(epochs, train_accs, label="Train Acc", color="royalblue")
    ax2.plot(epochs, val_accs, label="Val Acc", color="crimson")
    ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy (%)")
    ax2.set_title("Accuracy Curve"); ax2.legend(); ax2.grid(alpha=0.3)

    fig.tight_layout()
    fig.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"  -> saved {save_path}")


def plot_confusion(model, loader, class_names, device, save_path,
                   title="Confusion Matrix"):
    preds, labels = predict_all(model, loader, device)
    present = sorted(set(labels.tolist()))
    names = [class_names[i] for i in present]
    cm = confusion_matrix(labels, preds, labels=present)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

    fig, ax = plt.subplots(figsize=(8, 6))
    sns.heatmap(cm_norm, annot=True, fmt=".2f", cmap="Blues",
                xticklabels=names, yticklabels=names, ax=ax)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(title)
    fig.tight_layout()
    fig.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"  -> saved {save_path}")

    report = classification_report(
        labels, preds, labels=present, target_names=names, zero_division=0
    )
    print(report)
    return report


def plot_per_class_accuracy(model, loader, class_names, device, save_path):
    preds, labels = predict_all(model, loader, device)
    present = sorted(set(labels.tolist()))
    names = [class_names[i] for i in present]
    accs = []
    for c in present:
        mask = labels == c
        accs.append(100.0 * (preds[mask] == c).sum() / mask.sum())

    fig, ax = plt.subplots(figsize=(8, 5))
    bars = ax.bar(names, accs, color="steelblue", edgecolor="black")
    for b, a in zip(bars, accs):
        ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.5,
                f"{a:.1f}%", ha="center", va="bottom", fontsize=10)
    ax.set_ylim(0, 110)
    ax.set_ylabel("Accuracy (%)")
    ax.set_title("Per-Class Accuracy")
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()
    fig.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"  -> saved {save_path}")
```